

ShelbyFirst: Before & After Simulation

Tract-level visualization of Dental Visit Gap, Diabetes, and Obesity

Baseline anchored to slide 5 values. Post-ShelbyFirst reflects a data-informed 3-year projection.

```
In [1]: import pandas as pd
import numpy as np
import geopandas as gpd
import json
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')
```

1. Load Geospatial and IGS Data

```
In [ ]: # Load Tennessee tract shapefile and filter to Shelby County (FIPS 157)
gdf = gpd.read_file('data/raw/tl_2024_state_tracts/tl_2024_47_tract/tl_2024_47_tract.shp')
shelby_geo = gdf[gdf['COUNTYFP'] == '157'].copy()
shelby_geo = shelby_geo.to_crs(epsg=4326) # WGS84 for Plotly
print(f'Shelby County tracts (all): {len(shelby_geo)}')
```

```
In [ ]: # Load 96 Low-IGS tract panel (recovery period average)
panel = pd.read_parquet('data/processed/enrichment_tract_year_panel.parquet')
shelby_panel = panel[panel['geoid'].astype(str).str.startswith('47157')]

recovery_tracts = (
    shelby_panel[shelby_panel['analysis_period'] == 'recovery']
    .groupby('geoid')
    .agg(igs_total=('igs_total', 'mean'), poverty_rate=('poverty_rate', 'mean'))
    .reset_index()
)
recovery_tracts['geoid'] = recovery_tracts['geoid'].astype(str)
print(f'Low-IGS recovery tracts: {len(recovery_tracts)}')
print(f'IGS range: {recovery_tracts.igs_total.min():.1f} - {recovery_tracts.igs_tot
```

2. Build Tract-Level Baseline (Pre-ShelbyFirst)

Anchored to PDF slide 5 community-level averages:

Metric	Low-IGS	Avg-IGS	High-IGS
Dental Visit Gap	55.6%	43.1%	32.1%
Obesity	45.2%	43.1%	32.5%
Diabetes	21.6%	17.0%	11.3%

Each tract's value is estimated by linearly interpolating from its IGS score, with small random noise (seed-fixed) to create realistic spatial variation.

```
In [ ]: np.random.seed(42)

# IGS anchor points from PDF slide 5
# Format: (igs_low, val_at_low, igs_high, val_at_high)
anchors = {
    'dental_visit_gap': (20, 0.556, 56, 0.321),
    'obesity':          (20, 0.452, 56, 0.325),
    'diabetes':         (20, 0.216, 56, 0.113),
}

noise_scale = {'dental_visit_gap': 0.025, 'obesity': 0.020, 'diabetes': 0.012}

def interpolate_metric(igs, low_igs, val_low, high_igs, val_high, noise):
    """Linear interpolation with clipped noise."""
    t = np.clip((igs - low_igs) / (high_igs - low_igs), 0, 1)
    base = val_low + t * (val_high - val_low)
    return np.clip(base + noise, 0.01, 0.99)

for metric, (igs_l, v_l, igs_h, v_h) in anchors.items():
    noise = np.random.normal(0, noise_scale[metric], len(recovery_tracts))
    recovery_tracts[f'{metric}_before'] = interpolate_metric(
        recovery_tracts['igs_total'], igs_l, v_l, igs_h, v_h, noise
    )

print('Baseline means (should be close to slide 5 community avg ~35 IGS):')
for m in anchors:
    print(f' {m}: {recovery_tracts[f"{m}_before"].mean():.1%}')
```

3. Model ShelbyFirst 3-Year Impact

Logic:

- Priority tracts (IGS < 30, most vulnerable) receive the strongest impact
- Moderate tracts (IGS 30–40) see intermediate improvement
- Higher-IGS tracts see smaller but real gains from community spillovers
- Dental visit gap improves most (transportation friction directly addressed)
- Diabetes & obesity improve slower (chronic disease takes longer to shift)

```
In [ ]: # Reduction multipliers per metric based on ShelbyFirst intervention logic
# dental: transportation fix directly reduces missed appointments
# obesity: nutrition/garden programs + activity
# diabetes: longer lag; monitoring + prevention improve over 3 years

metric_reduction_caps = {
    'dental_visit_gap': 0.22, # up to 22% relative reduction in worst tracts
    'obesity':          0.10, # up to 10% relative reduction
    'diabetes':          0.08, # up to 8% relative reduction
}
```

```

def compute_reduction(igs_scores, metric, before_values):
    """
    Tracts with lower IGS scores are higher priority and get more ShelbyFirst
    resources. Reduction scales from max_reduction (at IGS=20) to ~30% of max
    (at IGS=56).
    """
    max_r = metric_reduction_caps[metric]
    # Linear scale: IGS=20 -> max_r, IGS=56 -> 0.30*max_r
    t = np.clip((igs_scores - 20) / (56 - 20), 0, 1)
    tract_reduction_rate = max_r * (1 - 0.70 * t)
    # Small random variation in uptake
    np.random.seed(99)
    uptake_noise = np.random.uniform(0.85, 1.15, len(igs_scores))
    tract_reduction_rate = tract_reduction_rate * uptake_noise
    return np.clip(before_values * (1 - tract_reduction_rate), 0.01, 0.99)

for metric in anchors:
    recovery_tracts[f'{metric}_after'] = compute_reduction(
        recovery_tracts['igs_total'],
        metric,
        recovery_tracts[f'{metric}_before']
    )
    delta = recovery_tracts[f'{metric}_before'].mean() - recovery_tracts[f'{metric}_after'].mean()
    print(f'{metric}: {recovery_tracts[f'{metric}_before'].mean():.1%} → {recovery_tracts[f'{metric}_after'].mean():.1%}')

```

4. Merge with Geometry

```

In [ ]: shelby_geo['GEOID'] = shelby_geo['GEOID'].astype(str)

# Merge: Low-IGS tracts get simulated values, rest get NaN (shown as grey)
sim_gdf = shelby_geo.merge(recovery_tracts, left_on='GEOID', right_on='geoid', how='left')

# Convert to GeoJSON for Plotly
geojson = json.loads(sim_gdf.to_json())
sim_gdf['id'] = sim_gdf.index.astype(str)

# Assign IDs to GeoJSON features
for i, feat in enumerate(geojson['features']):
    feat['id'] = str(i)

print(f'Total Shelby tracts: {len(sim_gdf)}')
print(f'Low-IGS tracts with simulation data: {sim_gdf["igs_total"].notna().sum()}')

```

5. Interactive Before & After Maps

```

In [ ]: METRIC_CONFIG = {
    'dental_visit_gap': {
        'label': 'Dental Visit Gap',
        'unit': '%',
        'colorscale': 'YlOrRd',
        'zmin': 0.28,
        'zmax': 0.62,
    }
}

```

```

        'fmt': '.1%',
    },
    'obesity': {
        'label': 'Adult Obesity Rate',
        'unit': '%',
        'colorscale': 'Oranges',
        'zmin': 0.28,
        'zmax': 0.50,
        'fmt': '.1%',
    },
    'diabetes': {
        'label': 'Diabetes Prevalence',
        'unit': '%',
        'colorscale': 'PuRd',
        'zmin': 0.09,
        'zmax': 0.24,
        'fmt': '.1%',
    },
}

CENTER = {'lat': 35.10, 'lon': -90.05}
ZOOM = 9.5

def make_before_after(metric_key):
    cfg = METRIC_CONFIG[metric_key]
    col_before = f'{metric_key}_before'
    col_after = f'{metric_key}_after'

    # Build hover text
    def hover(row, col):
        v = row[col]
        if pd.isna(v):
            return f'{row["NAME_SAD"]}<br>Not in low-IGS cluster'
        igs = row['igs_total']
        return (
            f"<b>{row['NAME_SAD']}</b><br>"
            f"{cfg['label']}: {v:.1%}<br>"
            f"IGS Score: {igs:.1f}<br>"
            f"Poverty Rate: {row['poverty_rate']:.1%}"
        )

    sim_gdf['hover_before'] = sim_gdf.apply(lambda r: hover(r, col_before), axis=1)
    sim_gdf['hover_after'] = sim_gdf.apply(lambda r: hover(r, col_after), axis=1)

    fig = make_subplots(
        rows=1, cols=2,
        subplot_titles=[
            f'<b>BEFORE</b> ShelbyFirst - {cfg["label"]}',
            f'<b>AFTER</b> ShelbyFirst (3-Year Projection) - {cfg["label"]}',
        ],
        specs=[[{'type': 'mapbox'}], [{'type': 'mapbox'}]],
        horizontal_spacing=0.02,
    )

    for col_idx, (col, hover_col, title_suffix) in enumerate(

```

```

[(col_before, 'hover_before', 'Before'),
 (col_after, 'hover_after', 'After')], start=1
):
    z_vals = pd.to_numeric(sim_gdf[col], errors='coerce').tolist()

    fig.add_trace(
        go.Choroplethmapbox(
            geojson=geojson,
            locations=sim_gdf['id'],
            z=z_vals,
            colorscale=cfg['colorscale'],
            zmin=cfg['zmin'],
            zmax=cfg['zmax'],
            marker_opacity=0.85,
            marker_line_width=0.4,
            marker_line_color='white',
            text=sim_gdf[hover_col],
            hovertemplate='%{text}<extra></extra>',
            showscale=(col_idx == 2),
            colorbar=dict(
                tickformat='.0%',
                title=cfg['label'],
                thickness=14,
                len=0.55,
                x=1.01,
            ) if col_idx == 2 else None,
        ),
        row=1, col=col_idx,
    )

# Aggregate summary annotation
mask = sim_gdf[col_before].notna()
avg_before = sim_gdf.loc[mask, col_before].mean()
avg_after = sim_gdf.loc[mask, col_after].mean()
delta_pp = (avg_before - avg_after) * 100
pct_change = (avg_after - avg_before) / avg_before * 100

fig.update_layout(
    title=dict(
        text=(
            f'<b>ShelbyFirst Impact Simulation - {cfg["label"]}</b><br>'
            f'<span style="font-size:13px; color:#555">'
            f'Low-IGS cluster avg: {avg_before:.1%} → {avg_after:.1%} '
            f'({delta_pp:.1f} pp reduction | {abs(pct_change):.1f}% relative im'
            f'</span>'
        ),
        x=0.5, xanchor='center', font_size=16,
    ),
    mapbox=dict(
        style='carto-positron', center=CENTER, zoom=ZOOM
    ),
    mapbox2=dict(
        style='carto-positron', center=CENTER, zoom=ZOOM
    ),
    height=600,
    margin=dict(l=10, r=80, t=90, b=10),

```

```

        paper_bgcolor='#fafafa',
    )

    return fig

print('Plot functions ready.')

```

5a. Dental Visit Gap

```

In [ ]: fig_dental = make_before_after('dental_visit_gap')
fig_dental.show()

```

5b. Adult Obesity Rate

```

In [ ]: fig_obesity = make_before_after('obesity')
fig_obesity.show()

```

5c. Diabetes Prevalence

```

In [ ]: fig_diabetes = make_before_after('diabetes')
fig_diabetes.show()

```

6. Combined Summary Chart

```

In [ ]: mask = sim_gdf['igs_total'].notna()

summary_rows = []
for metric_key, cfg in METRIC_CONFIG.items():
    before_vals = sim_gdf.loc[mask, f'{metric_key}_before']
    after_vals = sim_gdf.loc[mask, f'{metric_key}_after']
    summary_rows.append({
        'metric': cfg['label'],
        'before': before_vals.mean(),
        'after': after_vals.mean(),
        'delta_pp': (before_vals.mean() - after_vals.mean()) * 100,
        'pct_imp': (before_vals.mean() - after_vals.mean()) / before_vals.mean() *
    })

summary = pd.DataFrame(summary_rows)

fig_summary = go.Figure()

colors_before = ['#e05c5c', '#e07b30', '#9b59b6']
colors_after = ['#5ca8e0', '#5ce08a', '#5ce0d9']

x_labels = summary['metric'].tolist()

fig_summary.add_trace(go.Bar(
    name='Before ShelbyFirst',

```

```

x=x_labels,
y=(summary['before'] * 100).tolist(),
marker_color=colors_before,
text=[f'{v:.1f}%' for v in summary['before'] * 100],
textposition='outside',
width=0.3,
offset=-0.18,
))

fig_summary.add_trace(go.Bar(
    name='After ShelbyFirst (3-Yr)',
    x=x_labels,
    y=(summary['after'] * 100).tolist(),
    marker_color=colors_after,
    text=[f'{v:.1f}%' for v in summary['after'] * 100],
    textposition='outside',
    width=0.3,
    offset=0.18,
))

# Improvement annotations
for i, row in summary.iterrows():
    fig_summary.add_annotation(
        x=row['metric'],
        y=max(row['before'], row['after']) * 100 + 3.5,
        text=f'-{row["delta_pp"]:.1f} pp<br>({row["pct_imp"]:.0f}% ↓)',
        showarrow=False,
        font=dict(size=11, color='#2c7a2c'),
        align='center',
    )

fig_summary.update_layout(
    title=dict(
        text='<b>ShelbyFirst 3-Year Impact: Low-IGS Cluster Averages</b>',
        x=0.5, xanchor='center', font_size=17,
    ),
    barmode='overlay',
    yaxis=dict(title='Rate (%)', ticksuffix='%', range=[0, 75]),
    legend=dict(orientation='h', y=1.08, x=0.5, xanchor='center'),
    height=480,
    plot_bgcolor='white',
    paper_bgcolor='#fafafa',
    margin=dict(t=100, b=40),
)

fig_summary.show()
print(summary[['metric', 'before', 'after', 'delta_pp', 'pct_imp']].to_string(index=False))

```

7. Priority Tract Deep Dive (Whitehaven ZIPs Focus)

Tracts with IGS < 30 are the highest-priority intervention targets.

```

In [ ]: high_priority = sim_gdf[sim_gdf['igs_total'] < 30].copy()
print(f'Highest-priority tracts (IGS < 30): {len(high_priority)}')

```

```

rows = []
for _, tract in high_priority.iterrows():
    rows.append({
        'Tract': tract['NAMELSAD'],
        'IGS Score': round(tract['igs_total'], 1),
        'Dental Gap Before': f"{tract['dental_visit_gap_before']:.1%}",
        'Dental Gap After': f"{tract['dental_visit_gap_after']:.1%}",
        'Obesity Before': f"{tract['obesity_before']:.1%}",
        'Obesity After': f"{tract['obesity_after']:.1%}",
        'Diabetes Before': f"{tract['diabetes_before']:.1%}",
        'Diabetes After': f"{tract['diabetes_after']:.1%}",
    })

priority_df = pd.DataFrame(rows).sort_values('IGS Score')
print(priority_df.to_string(index=False))

```

8. Export Interactive Maps to HTML

```

In [ ]: import os
out_dir = 'data/processed'

fig_dental.write_html(os.path.join(out_dir, 'shelbyfirst_dental_simulation.html'))
fig_obesity.write_html(os.path.join(out_dir, 'shelbyfirst_obesity_simulation.html'))
fig_diabetes.write_html(os.path.join(out_dir, 'shelbyfirst_diabetes_simulation.html'))
fig_summary.write_html(os.path.join(out_dir, 'shelbyfirst_summary_chart.html'))

print('Exported:')
for f in ['shelbyfirst_dental_simulation.html', 'shelbyfirst_obesity_simulation.html',
         'shelbyfirst_diabetes_simulation.html', 'shelbyfirst_summary_chart.html']:
    print(' ', os.path.join(out_dir, f))

```